

The Builder's Guide to Skills

For the hands-on. How to write, test and maintain skills; how the workspace fits together; connectors, permissions, and keeping the numbers honest.

The promise that makes this approachable: a skill is just a Markdown file. If you can write an email briefing a junior analyst, you can write one — and Claude will write the first draft for you. Nothing here requires a software background.

1 • What a skill is

A skill is a folder with one file that teaches Claude how to do a recurring task *your* way. Claude reads its short description, and when a task matches, it loads the full instructions — and only then. That "load only when needed" design is deliberate: it keeps Claude's attention focused (see §7).

The one gotcha that breaks everything: a skill must live at `.claude/skills/<skill-name>/SKILL.md` **inside the working folder**. Anywhere else and it will not trigger. This is the single most common setup mistake.

2 • Anatomy of a SKILL.md

```
---
name: jse-mining
description: >
  Sector skill for analysing JSE mining companies. Use whenever the user
  asks to analyse or compare a miner's results, or mentions Anglo, BHP,
  Sasol, Gold Fields, Exxaro... Reads documents already downloaded to
  companies/[slug]/; cites every figure to document and page.
---

# JSE Mining & Resources Analyst

## Purpose
Produces standardised analysis of mining results from local documents...

## Metrics to extract (current AND prior period, with change)
- Revenue, EBITDA & margin, HEPS, free cash flow, dividend
- Net debt and net-debt-to-EBITDA (the key leverage gauge)
- Production volumes and unit cost vs guidance

## Rules
- Numbers are sacred; never round or guess. Derived figures marked (e).
- Cite every figure to document and page. Prior-period comparison always.
- Missing disclosure → "not disclosed", never papered over.
```

Three parts, all plain English:

Part	Job	Tip
name	The folder/skill identifier	kebab-case, e.g. jse-mining

description	The trigger — how Claude decides <i>when</i> to use it	List the phrases people actually say. This is the most important line.
Instructions	What to do — sections, order, format	Write it like a brief to a new hire
Rules	Guardrails — what never to do	Be explicit: "never invent", "always cite"

3 • The easy way: ask Claude to write it

You rarely write the file by hand. Describe what you want and Claude scaffolds it:

```
"Create a skill for analysing JSE mining results: revenue, EBITDA and margin,
HEPS, net debt and net-debt-to-EBITDA, free cash flow and dividends, always vs
the prior period, every figure cited. Flag commodity-price and rand sensitivity.
Never invent a number; write 'not disclosed' where missing."
```

Then run the loop that is the whole craft:

1. **Generate** — Claude writes the `SKILL.md`.
2. **Read** — open it; it's a document. Edit any line you'd phrase differently.
3. **Test** — point it at a real result and check the output.
4. **Refine** — in plain English: "also flag any company in a closed period." It updates the skill.

4 • What makes a good skill

- **One job.** "Analyse mining results" beats "handle all research".
- **A described trigger.** Include the casual phrasings — that's how it fires automatically.
- **Your standards encoded.** Numbers sacred, cite everything, always pcq. House style becomes automatic.
- **Honest guardrails.** Tell it what to do when data is missing.

5 • How the workspace fits together

```
JSE Financial Reports/
├─ CLAUDE.md
├─ manifest.json
├─ tools/manifest.py
├─ .claude/skills/
│   └─ jse-analyst/SKILL.md
│   └─ jse-mining/SKILL.md
└─ companies/<slug>/
    └─ annual-reports/  interim-reports/  sens-announcements/ ...
```

← the working folder (the research root)
the "employee handbook": identity, principles, rules
GENERATED index of every document — never hand-edit
the generator/validator for the manifest
all skills live here (note the path!)

downloaded docs, one folder per company

CLAUDE.md is loaded at the start of every session — it's where the fund's principles live (numbers are sacred, source everything, prior-period always). Edit it to change how Claude behaves across every skill.

The manifest is generated, never authored. The source of truth is each `companies/<slug>/company.json` plus the files on disk. To change the index, change a `company.json` (or use the helper) and run `python3 tools/manifest.py rebuild`, then `validate`. Hand-editing `manifest.json` is what caused past corruption — don't.

6 · Connectors (MCP) — plugging into your systems

Connectors let Claude reach your tools. They're enabled once by an admin, then Claude can cite internal material alongside public filings.

Connector	What it adds
SharePoint	Read the fund's internal models and notes in place (e.g. <code>/Research/[Company]/</code>). For AllWeather, SharePoint also becomes the system of record — see the architecture note below.
Outlook	Broker research and results notifications received by email
Teams	Flag material findings to the desk
Bloomberg	Pricing and consensus, where licensed

Recommended Host the working folder in a SharePoint library synced locally via OneDrive: Claude works on fast local files, the team sees the same documents, and versioning/retention come free. Start with **one writer** (the pilot analyst) to avoid OneDrive sync conflicts; everyone else reads via SharePoint.

7 · Keeping the numbers honest

Three properties of the model, and the discipline each one demands:

Property	Why it matters	The discipline
Finite attention ("dumb zone")	Overload the context and quality drops	One job per skill; load only what's needed
Can be confidently wrong	Fluent ≠ correct	Cite every figure; spot-check the page
Non-deterministic	Same prompt, slightly different wording each run	Send exact, repeatable maths to code , not the model

Deterministic vs non-deterministic — the rule of thumb. Use the model for judgement, drafting and synthesis, where a little variation is fine or even useful. Use **code** for anything that must reconcile to the same answer every time — totals, ratios, the manifest. The workspace already does this: `tools/manifest.py` computes in Python, not freehand by the model.

8 · Permissions & the browser

- **Folder-scoped.** Claude reads and writes only the working folder you grant — nothing else on the machine.
- **Explicit prompts.** File, command and web actions sit behind a prompt; you approve per-action or per-session. Nothing happens silently.
- **Claude in Chrome.** Some IR sites block plain fetches; the Chrome extension lets Claude drive a real, *visible* browser tab to navigate and download. You watch every action.
- **Admin controls.** Org owners govern web access and connectors centrally (Admin settings → Capabilities).

9 · Maintenance

- **Sites change.** If a download breaks, the workspace re-discovers the source and repairs the company's skill — self-healing by design. If it can't, ask Claude to re-map the IR page.
- **Skills are text.** Version them with the rest of the folder (git, or SharePoint versioning). A change is a diff you can read.

- **Review descriptions.** If a skill stops triggering, its `description` probably needs the new phrasing people are using.
- Run `validate` after any change to a `company.json` or after dropping files in a company folder.

10 · Troubleshooting

Symptom	Likely cause	Fix
Skill never triggers	Wrong location	Move to <code>.claude/skills/<name>/SKILL.md</code> inside the working folder
Downloads fail on an IR site	Site blocks plain fetches	Connect the Chrome extension; Claude falls back to the browser
<code>manifest.json</code> looks wrong	Hand-edited or stale	Never hand-edit; <code>rebuild</code> then <code>validate</code>
Claude can't see a file	Outside the working folder	Move/copy it in, or connect the relevant MCP connector
SharePoint files missing locally	OneDrive set to online-only	Mark the library "Always keep on this device"

Companion Non-technical colleagues should start with **The Everyday Guide**. Full bootstrapping steps are in `setup-guide.md`; the skill recipe in `skill-creation-guide.md`. *Informational use only; not investment advice.*